

Seance 5

Templates et Programmation Generique
Ecrire du code qui marche avec N'IMPORTE QUEL type

Cours de renforcement C++

Table des matières

1	Introduction : Le probleme de la duplication de code	3
1.1	Rappel des seances precedentes	3
1.2	Le probleme sans templates	3
1.3	Comment ca marche ?	4
2	Templates de fonctions	5
2.1	Syntaxe de base	5
2.2	Utilisation	5
2.3	Template avec plusieurs types	5
2.4	Template avec type de retour different	6
2.5	Exemple concret : Fonction swap generique	6
3	Templates de classes	7
3.1	Le probleme	7
3.2	Exemple complet : Classe Pair	8
3.3	Methodes template dans une classe	8
3.4	Exemple realiste : Classe Array generique	9
4	Specialisation de templates	11
4.1	Le probleme	11
4.2	Specialisation complete	11
4.3	Exemple : Specialisation pour les pointeurs	12
4.4	Specialisation partielle	12
5	Templates avec valeurs (non-type templates)	14
5.1	Le concept	14
5.2	Exemple : Matrice de taille fixe	14
6	Templates et smart pointers	16
6.1	Recreer unique_ptr (simplifie)	16
6.2	Templates et interfaces	17
7	Exercices pratiques	19
7.1	Exercice 1 : Fonction min generique (facile)	19
7.2	Exercice 2 : Classe Stack generique (moyen)	19
7.3	Exercice 3 : Fonction print generique (moyen)	20
7.4	Exercice 4 : Optional generique simplifie (difficile)	21

8	Templates variadiques (C++11+)	23
8.1	Le concept	23
8.2	Recursion template	23
8.3	Exemple : make_unique	23
8.4	Fold expressions (C++17)	24
9	SFINAE et concepts (aperçu)	25
9.1	Le problème	25
9.2	enable_if (C++11)	25
9.3	Concepts (C++20)	25
10	Pièges et bonnes pratiques	26
10.1	Piège 1 : Templates dans les headers	26
10.2	Piège 2 : Erreurs de compilation incompréhensibles	26
10.3	Piège 3 : Bloat de code	26
10.4	Bonne pratique 1 : Utiliser auto	26
10.5	Bonne pratique 2 : Template alias	27
11	Quiz de validation	28
11.1	Questions	28
11.2	Reponses	28

1 Introduction : Le probleme de la duplication de code

1.1 Rappel des seances precedentes

Dans la Seance 4, nous avons vu les smart pointers qui gerent automatiquement la memoire. Mais regardez ce code :

```

1 // Smart pointer pour int
2 std::unique_ptr<int> ptrInt = std::make_unique<int>(42);
3
4 // Smart pointer pour string
5 std::unique_ptr<std::string> ptrStr =
6     std::make_unique<std::string>("hello");
7
8 // Smart pointer pour notre classe
9 std::unique_ptr<Widget> ptrWidget =
10     std::make_unique<Widget>();

```

Question importante

Comment unique_ptr peut-il marcher avec int, string, Widget, et N'IMPORTE QUEL type ?

Est-ce qu'il y a une version de unique_ptr pour chaque type possible ? NON !

Reponse : unique_ptr est un **template**. Il est ecrit UNE SEULE FOIS de facon gen-
erique.

1.2 Le probleme sans templates

Imaginez que vous voulez creer une fonction qui retourne le maximum de deux valeurs.

Sans templates (horrible) :

```

1 int maxInt(int a, int b) {
2     return (a > b) ? a : b;
3 }
4
5 double maxDouble(double a, double b) {
6     return (a > b) ? a : b;
7 }
8
9 std::string maxString(const std::string& a, const std::string& b) {
10     return (a > b) ? a : b;
11 }
12
13 // Et ainsi de suite pour CHAQUE type...
14 // DUPLICATION MASSIVE !

```

Avec templates (elegant) :

```

1 template<typename T>
2 T max(T a, T b) {
3     return (a > b) ? a : b;
4 }
5
6 // UNE SEULE fonction qui marche avec TOUS les types !
7 int i = max(10, 20);           // T = int
8 double d = max(3.14, 2.71);    // T = double
9 std::string s = max("abc", "xyz"); // T = const char*

```

Le pouvoir des templates

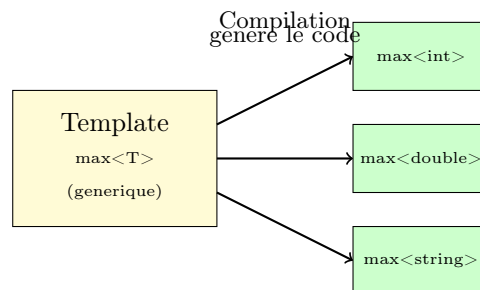
Un seul code, infinite de types !

Les templates permettent d'ecrire du code **generique** qui s'adapte automatiquement au type utilise.

Metaphore : C'est comme un moule a gateau. Un seul moule, mais vous pouvez faire des gateaux au chocolat, a la vanille, aux fruits, etc.

1.3 Comment ca marche ?

Les templates ne sont PAS du code executable. Ce sont des **recettes** pour generer du code.



Processus :

1. Vous ecrivez le template (recette generale)
2. Le compilateur voit `max(10, 20)`
3. Il genere automatiquement la version `max<int>`
4. Vous appelez `max(3.14, 2.71)`
5. Il genere automatiquement la version `max<double>`
6. Etc.

C'est fait a la COMPILATION, pas a l'execution ! Donc zero cout en performance.

2 Templates de fonctions

2.1 Syntaxe de base

```

1 template<typename T>
2 T max(T a, T b) {
3     return (a > b) ? a : b;
4 }

```

Decomposition

template<typename T> : Declaration du template
typename T : T est un type generique (comme une variable, mais pour les types)
T max(...) : La fonction retourne un T
T a, T b : Les parametres sont de type T
Note : On peut aussi ecrire `template<class T>`. C'est exactement pareil que `typename`.

2.2 Utilisation

```

1 int main() {
2     // Le compilateur DEDUIT le type automatiquement
3     int i = max(10, 20);           // T = int
4     double d = max(3.14, 2.71);   // T = double
5
6     // On peut aussi specifier explicitement le type
7     auto result = max<int>(10, 20);
8
9     // ERREUR : les deux parametres doivent etre du meme type
10    // auto bad = max(10, 3.14); // int vs double !
11 }

```

Deduction de type

Le compilateur est intelligent ! Il regarde les arguments et deduit automatiquement le type T.

max(10, 20) → 10 et 20 sont des int → T = int

max(3.14, 2.71) → 3.14 et 2.71 sont des double → T = double

Attention : Tous les T doivent etre du MEME type !

2.3 Template avec plusieurs types

```

1 template<typename T, typename U>
2 void afficher(T premier, U second) {
3     std::cout << "Premier : " << premier << "\n";
4     std::cout << "Second : " << second << "\n";
5 }
6
7 int main() {
8     afficher(42, "hello");           // T=int, U=const char*
9     afficher(3.14, 100);             // T=double, U=int
10    afficher("abc", "xyz");          // T=const char*, U=const char*
11 }

```

2.4 Template avec type de retour different

```
1  template<typename T>
2  std::string toString(T value) {
3      // Convertit n'importe quel type en string
4      std::ostringstream oss;
5      oss << value;
6      return oss.str();
7  }
8
9  int main() {
10     std::string s1 = toString(42);           // "42"
11     std::string s2 = toString(3.14);        // "3.14"
12     std::string s3 = toString(true);        // "1"
13 }
```

2.5 Exemple concret : Fonction swap generique

```
1  template<typename T>
2  void swap(T& a, T& b) {
3      T temp = a;
4      a = b;
5      b = temp;
6  }
7
8  int main() {
9      int x = 10, y = 20;
10     swap(x, y);
11     std::cout << x << " " << y << "\n"; // 20 10
12
13     std::string s1 = "hello", s2 = "world";
14     swap(s1, s2);
15     std::cout << s1 << " " << s2 << "\n"; // world hello
16
17     // Marche avec N'IMPORTE QUEL type !
18 }
```

3 Templates de classes

3.1 Le probleme

Imaginez que vous voulez creer une boite qui peut contenir n'importe quel type d'objet.

Sans templates :

```

1 class BoxInt {
2     int value;
3 public:
4     BoxInt(int v) : value(v) {}
5     int get() { return value; }
6 };
7
8 class BoxDouble {
9     double value;
10 public:
11     BoxDouble(double v) : value(v) {}
12     double get() { return value; }
13 };
14
15 // Une classe par type... horrible !

```

Avec templates :

```

1 template<typename T>
2 class Box {
3 private:
4     T value;
5
6 public:
7     Box(T v) : value(v) {}
8
9     T get() const {
10         return value;
11     }
12
13     void set(T v) {
14         value = v;
15     }
16 };
17
18 // UNE SEULE classe pour TOUS les types !
19 Box<int> boxInt(42);
20 Box<std::string> boxStr("hello");
21 Box<double> boxDouble(3.14);

```

Difference avec les fonctions

Fonctions template : Le type est souvent deduit automatiquement

Classes template : On DOIT specifier le type explicitement

```

1 max(10, 20);           // OK : deduit T = int
2 Box box(42);           // ERREUR ! Quel est T ?
3 Box<int> box(42);       // OK : T = int explicite

```

Exception : Depuis C++17, on peut parfois deduire le type grace aux guides de deduction.

3.2 Exemple complet : Classe Pair

```

1  template<typename T1, typename T2>
2  class Pair {
3  private:
4      T1 first;
5      T2 second;
6
7  public:
8      Pair(T1 f, T2 s) : first(f), second(s) {}
9
10     T1 getFirst() const { return first; }
11     T2 getSecond() const { return second; }
12
13     void setFirst(T1 f) { first = f; }
14     void setSecond(T2 s) { second = s; }
15
16     void afficher() const {
17         std::cout << "(" << first << ", " << second << ")\n";
18     }
19 };
20
21 int main() {
22     Pair<int, std::string> p1(42, "reponse");
23     p1.afficher(); // (42, reponse)
24
25     Pair<std::string, double> p2("pi", 3.14159);
26     p2.afficher(); // (pi, 3.14159)
27
28     Pair<int, int> p3(10, 20);
29     p3.afficher(); // (10, 20)
30 }

```

3.3 Methodes template dans une classe

```

1  template<typename T>
2  class Container {
3  private:
4      std::vector<T> data;
5
6  public:
7      void add(T value) {
8          data.push_back(value);
9      }
10
11     // Methode template DANS une classe template
12     template<typename U>
13     void addConverted(U value) {
14         data.push_back(static_cast<T>(value));
15     }
16
17     void afficher() const {
18         for (const auto& item : data) {
19             std::cout << item << " ";
20         }
21         std::cout << "\n";
22     }

```

```

23 };
24
25 int main() {
26     Container<int> c;
27     c.add(10);           // T = int
28     c.addConverted(3.14); // U = double, converti en int
29     c.addConverted('A');  // U = char, converti en int
30     c.afficher();        // 10 3 65
31 }

```

3.4 Exemple realiste : Classe Array generique

```

1  template<typename T>
2  class Array {
3  private:
4      T* data;
5      size_t taille;
6
7  public:
8      // Constructeur
9      Array(size_t t) : taille(t) {
10         data = new T[taille];
11         std::cout << "Array de " << taille << " elements cree\n";
12     }
13
14     // Destructeur
15     ~Array() {
16         delete[] data;
17         std::cout << "Array detruit\n";
18     }
19
20     // Copie interdite (pour simplifier)
21     Array(const Array&) = delete;
22     Array& operator=(const Array&) = delete;
23
24     // Acces par index
25     T& operator[](size_t index) {
26         if (index >= taille) {
27             throw std::out_of_range("Index hors limites");
28         }
29         return data[index];
30     }
31
32     const T& operator[](size_t index) const {
33         if (index >= taille) {
34             throw std::out_of_range("Index hors limites");
35         }
36         return data[index];
37     }
38
39     size_t size() const {
40         return taille;
41     }
42
43     void fill(T value) {
44         for (size_t i = 0; i < taille; ++i) {
45             data[i] = value;

```

```
46     }
47 }
48 };
49
50 int main() {
51     Array<int> intArray(5);
52     intArray.fill(42);
53     std::cout << intArray[0] << "\n"; // 42
54
55     Array<std::string> strArray(3);
56     strArray[0] = "hello";
57     strArray[1] = "world";
58     std::cout << strArray[0] << "\n"; // hello
59 }
```

4 Specialisation de templates

4.1 Le probleme

Parfois, le code generique ne marche pas bien pour un type specifique. On veut une version speciale.

Exemple : Une classe pour afficher des valeurs

```
1 template<typename T>
2 class Printer {
3 public:
4     static void print(T value) {
5         std::cout << value << "\n";
6     }
7 };
8
9 // Ca marche pour int, double, string...
10 Printer<int>::print(42);
11 Printer<std::string>::print("hello");
12
13 // Mais pour bool, on voudrait afficher "true"/"false"
14 // au lieu de "1"/"0"
15 Printer<bool>::print(true); // Affiche : 1
```

4.2 Specialisation complete

On peut creer une version speciale pour un type precis :

```
1 // Template generique
2 template<typename T>
3 class Printer {
4 public:
5     static void print(T value) {
6         std::cout << value << "\n";
7     }
8 };
9
10 // Specialisation pour bool
11 template<>
12 class Printer<bool> {
13 public:
14     static void print(bool value) {
15         std::cout << (value ? "true" : "false") << "\n";
16     }
17 };
18
19 int main() {
20     Printer<int>::print(42);           // 42
21     Printer<bool>::print(true);        // true
22     Printer<bool>::print(false);       // false
23 }
```

Syntaxe de specialisation

`template<>` : Specialisation (liste de parametres vide)
`class Printer<bool>` : Version speciale pour bool
 Le compilateur choisit automatiquement la version appropriée :

- `Printer<int>` → version generique
- `Printer<bool>` → version specialisee

4.3 Exemple : Specialisation pour les pointeurs

```

1  // Template generique
2  template<typename T>
3  class Storage {
4  private:
5      T value;
6  public:
7      Storage(T v) : value(v) {}
8      T get() { return value; }
9  };
10
11 // Specialisation pour les pointeurs
12 template<typename T>
13 class Storage<T*> {
14 private:
15     T* value;
16 public:
17     Storage(T* v) : value(v) {}
18
19     T* get() { return value; }
20
21     // Methode speciale pour les pointeurs
22     T& dereference() { return *value; }
23
24     ~Storage() {
25         // On pourrait delete value ici si on voulait
26     }
27 };
28
29 int main() {
30     Storage<int> s1(42);
31     std::cout << s1.get() << "\n"; // 42
32
33     int value = 100;
34     Storage<int*> s2(&value);
35     std::cout << s2.dereference() << "\n"; // 100
36 }

```

4.4 Specialisation partielle

On peut specialiser seulement certains parametres :

```

1  // Template avec deux parametres
2  template<typename T, typename U>
3  class Pair {
4  public:

```

```
5     void afficher() {
6         std::cout << "Pair generique\n";
7     }
8 };
9
10 // Specialisation partielle : les deux types sont identiques
11 template<typename T>
12 class Pair<T, T> {
13 public:
14     void afficher() {
15         std::cout << "Pair avec types identiques\n";
16     }
17 };
18
19 int main() {
20     Pair<int, double> p1;
21     p1.afficher(); // Pair generique
22
23     Pair<int, int> p2;
24     p2.afficher(); // Pair avec types identiques
25 }
```

5 Templates avec valeurs (non-type templates)

5.1 Le concept

Les templates peuvent aussi prendre des VALEURS constantes, pas seulement des types !

```

1  template<typename T, size_t N>
2  class FixedArray {
3  private:
4      T data[N]; // Tableau de taille FIXE
5
6  public:
7      T& operator[](size_t index) {
8          return data[index];
9      }
10
11     size_t size() const {
12         return N;
13     }
14 };
15
16 int main() {
17     FixedArray<int, 5> arr1; // Tableau de 5 int
18     FixedArray<int, 10> arr2; // Tableau de 10 int
19     FixedArray<double, 3> arr3; // Tableau de 3 double
20
21     // arr1 et arr2 sont des TYPES DIFFERENTS !
22     // Meme si tous deux sont FixedArray<int, ...>
23 }
```

Avantages

Zero overhead : La taille est connue a la compilation

Pas d'allocation dynamique : Le tableau est sur la pile

Type-safe : FixedArray<int, 5> et FixedArray<int, 10> sont des types incompatibles

C'est exactement ce que fait std::array !

```

1  std::array<int, 5> arr; // Template avec type ET taille
```

5.2 Exemple : Matrice de taille fixe

```

1  template<typename T, size_t ROWS, size_t COLS>
2  class Matrix {
3  private:
4      T data[ROWS][COLS];
5
6  public:
7      Matrix() {
8          // Initialiser a zero
9          for (size_t i = 0; i < ROWS; ++i) {
10             for (size_t j = 0; j < COLS; ++j) {
11                 data[i][j] = T();
12             }
13         }
14     }
15 }
```

```
16     T& at(size_t row, size_t col) {
17         if (row >= ROWS || col >= COLS) {
18             throw std::out_of_range("Index hors limites");
19         }
20         return data[row][col];
21     }
22
23     size_t rows() const { return ROWS; }
24     size_t cols() const { return COLS; }
25
26     void afficher() const {
27         for (size_t i = 0; i < ROWS; ++i) {
28             for (size_t j = 0; j < COLS; ++j) {
29                 std::cout << data[i][j] << " ";
30             }
31             std::cout << "\n";
32         }
33     }
34 };
35
36 int main() {
37     Matrix<int, 3, 3> mat;
38     mat.at(0, 0) = 1;
39     mat.at(1, 1) = 2;
40     mat.at(2, 2) = 3;
41     mat.afficher();
42     // 1 0 0
43     // 0 2 0
44     // 0 0 3
45 }
```

6 Templates et smart pointers

6.1 Recreer unique_ptr (simplifie)

Maintenant qu'on connaît les templates, on peut comprendre comment unique_ptr est implémenté!

```

1  template<typename T>
2  class SimpleUniquePtr {
3  private:
4      T* ptr;
5
6  public:
7      // Constructeur
8      explicit SimpleUniquePtr(T* p = nullptr) : ptr(p) {}
9
10     // Destructeur : libere automatiquement
11     ~SimpleUniquePtr() {
12         delete ptr;
13     }
14
15     // Interdire la copie
16     SimpleUniquePtr(const SimpleUniquePtr&) = delete;
17     SimpleUniquePtr& operator=(const SimpleUniquePtr&) = delete;
18
19     // Move constructor
20     SimpleUniquePtr(SimpleUniquePtr&& other) noexcept : ptr(other.ptr) {
21         other.ptr = nullptr;
22     }
23
24     // Move assignment
25     SimpleUniquePtr& operator=(SimpleUniquePtr&& other) noexcept {
26         if (this != &other) {
27             delete ptr;
28             ptr = other.ptr;
29             other.ptr = nullptr;
30         }
31         return *this;
32     }
33
34     // Operateurs
35     T& operator*() const { return *ptr; }
36     T* operator->() const { return ptr; }
37     T* get() const { return ptr; }
38
39     // Release : abandonner la propriete
40     T* release() {
41         T* temp = ptr;
42         ptr = nullptr;
43         return temp;
44     }
45
46     // Reset : changer de pointeur
47     void reset(T* p = nullptr) {
48         delete ptr;
49         ptr = p;
50     }
51 };
52

```

```

53 // Fonction make (simplifie)
54 template<typename T, typename... Args>
55 SimpleUniquePtr<T> makeSimpleUnique(Args&&... args) {
56     return SimpleUniquePtr<T>(new T(std::forward<Args>(args)...));
57 }
58
59 int main() {
60     SimpleUniquePtr<int> ptr = makeSimpleUnique<int>(42);
61     std::cout << *ptr << "\n"; // 42
62
63     SimpleUniquePtr<std::string> str =
64         makeSimpleUnique<std::string>("hello");
65     std::cout << *str << "\n"; // hello
66 }

```

Comment ça marche

SimpleUniquePtr<int> genere une classe qui gere un int*
 SimpleUniquePtr<string> genere une classe qui gere un string*
 Le meme code template genere des classes differentes pour chaque type!

6.2 Templates et interfaces

Combiner templates et interfaces :

```

1  template<typename T>
2  class Repository {
3  public:
4      virtual void save(const T& item) = 0;
5      virtual T load(int id) = 0;
6      virtual std::vector<T> loadAll() = 0;
7      virtual ~Repository() = default;
8  };
9
10 class User {
11     std::string nom;
12     int age;
13 public:
14     User(const std::string& n, int a) : nom(n), age(a) {}
15     // ...
16 };
17
18 class UserRepository : public Repository<User> {
19 private:
20     std::map<int, User> data;
21     int nextId = 1;
22
23 public:
24     void save(const User& user) override {
25         data[nextId++] = user;
26     }
27
28     User load(int id) override {
29         return data.at(id);
30     }
31
32     std::vector<User> loadAll() override {

```

```
33     std::vector<User> result;
34     for (const auto& pair : data) {
35         result.push_back(pair.second);
36     }
37     return result;
38 }
39 };
```

7 Exercices pratiques

7.1 Exercice 1 : Fonction min generique (facile)

Enonce

Creez une fonction template `min` qui retourne le minimum de deux valeurs de n'importe quel type.
Testez avec `int`, `double`, et `string`.

Solution :

```
1 template<typename T>
2 T min(T a, T b) {
3     return (a < b) ? a : b;
4 }
5
6 int main() {
7     std::cout << min(10, 20) << "\n";           // 10
8     std::cout << min(3.14, 2.71) << "\n";       // 2.71
9     std::cout << min("xyz", "abc") << "\n";     // abc
10 }
```

7.2 Exercice 2 : Classe Stack generique (moyen)

Enonce

Creez une classe template `Stack<T>` qui implemente une pile avec :

- `push(value)` : empiler
- `pop()` : depiler et retourner la valeur
- `top()` : voir le sommet sans depiler
- `empty()` : verifier si vide
- `size()` : nombre d'elements

Utilisez `std::vector` en interne.

Solution :

```
1 template<typename T>
2 class Stack {
3 private:
4     std::vector<T> data;
5
6 public:
7     void push(const T& value) {
8         data.push_back(value);
9     }
10
11     T pop() {
12         if (empty()) {
13             throw std::runtime_error("Stack vide");
14         }
15         T value = data.back();
16         data.pop_back();
17         return value;
18     }
19 }
```

```

19
20     const T& top() const {
21         if (empty()) {
22             throw std::runtime_error("Stack vide");
23         }
24         return data.back();
25     }
26
27     bool empty() const {
28         return data.empty();
29     }
30
31     size_t size() const {
32         return data.size();
33     }
34 };
35
36 int main() {
37     Stack<int> intStack;
38     intStack.push(10);
39     intStack.push(20);
40     intStack.push(30);
41
42     std::cout << intStack.pop() << "\n"; // 30
43     std::cout << intStack.top() << "\n"; // 20
44     std::cout << intStack.size() << "\n"; // 2
45
46     Stack<std::string> strStack;
47     strStack.push("hello");
48     strStack.push("world");
49     std::cout << strStack.pop() << "\n"; // world
50 }

```

7.3 Exercice 3 : Fonction print generique (moyen)

Enonce

Creez une fonction template `print` qui affiche n'importe quel conteneur (vector, list, etc.) entre crochets, separes par des virgules.

Exemple : [1, 2, 3, 4, 5]

Indice : Utilisez un itérateur generique.

Solution :

```

1 template<typename Container>
2 void print(const Container& container) {
3     std::cout << "[";
4     auto it = container.begin();
5     if (it != container.end()) {
6         std::cout << *it;
7         ++it;
8     }
9     for (; it != container.end(); ++it) {
10        std::cout << ", " << *it;
11    }
12    std::cout << "]\n";
13 }

```

```

14
15 int main() {
16     std::vector<int> vec = {1, 2, 3, 4, 5};
17     print(vec); // [1, 2, 3, 4, 5]
18
19     std::list<std::string> lst = {"hello", "world"};
20     print(lst); // [hello, world]
21
22     std::set<double> s = {3.14, 2.71, 1.41};
23     print(s); // [1.41, 2.71, 3.14]
24 }

```

7.4 Exercice 4 : Optional generique simplifie (difficile)

Enonce

Creez une classe template `Optional<T>` qui peut contenir une valeur ou rien (comme `std::optional`).

Methodes :

- `hasValue()` : retourne true si contient une valeur
- `value()` : retourne la valeur (throw si vide)
- `valueOr(defaultValue)` : retourne la valeur ou la valeur par default

Solution :

```

1  template<typename T>
2  class Optional {
3  private:
4      bool present;
5      T data;
6
7  public:
8      // Constructeur vide
9      Optional() : present(false), data() {}
10
11     // Constructeur avec valeur
12     Optional(const T& value) : present(true), data(value) {}
13
14     bool hasValue() const {
15         return present;
16     }
17
18     const T& value() const {
19         if (!present) {
20             throw std::runtime_error("Optional vide");
21         }
22         return data;
23     }
24
25     T& value() {
26         if (!present) {
27             throw std::runtime_error("Optional vide");
28         }
29         return data;
30     }
31 }

```

```
32     T valueOr(const T& defaultValue) const {
33         return present ? data : defaultValue;
34     }
35
36     void reset() {
37         present = false;
38     }
39
40     void set(const T& value) {
41         data = value;
42         present = true;
43     }
44 };
45
46 int main() {
47     Optional<int> opt1(42);
48     std::cout << opt1.value() << "\n"; // 42
49
50     Optional<int> opt2;
51     std::cout << opt2.valueOr(99) << "\n"; // 99
52
53     try {
54         std::cout << opt2.value() << "\n";
55     } catch (const std::exception& e) {
56         std::cout << "Erreur : " << e.what() << "\n";
57     }
58 }
```

8 Templates variadiques (C++11+)

8.1 Le concept

Les templates variadiques permettent de prendre un nombre VARIABLE de parametres de types VARIABLES.

```

1 // Fonction qui affiche n'importe quel nombre d'arguments
2 template<typename... Args>
3 void print(Args... args) {
4     // Comment traiter un nombre variable d'arguments ?
5 }
6
7 print(42);                // 1 argument
8 print(42, "hello");       // 2 arguments
9 print(42, "hello", 3.14); // 3 arguments

```

8.2 Recursion template

La technique classique est d'utiliser la recursion :

```

1 // Cas de base : 0 argument
2 void print() {
3     std::cout << "\n";
4 }
5
6 // Cas recursif : au moins 1 argument
7 template<typename T, typename... Args>
8 void print(T premier, Args... reste) {
9     std::cout << premier << " ";
10    print(reste...); // Appel recursif
11 }
12
13 int main() {
14     print(42, "hello", 3.14, true);
15     // Affiche : 42 hello 3.14 1
16 }

```

Comment ca marche

Appel : print(42, "hello", 3.14)

Expansion :

1. print(42, "hello", 3.14) → affiche 42, appelle print("hello", 3.14)
2. print("hello", 3.14) → affiche "hello", appelle print(3.14)
3. print(3.14) → affiche 3.14, appelle print()
4. print() → cas de base, affiche newline

C'est fait a la COMPILATION! Le compilateur genere toutes ces fonctions.

8.3 Exemple : make_unique

C'est exactement comme ca que make_unique est implemente!

```

1 template<typename T, typename... Args>
2 std::unique_ptr<T> make_unique(Args&&... args) {
3     return std::unique_ptr<T>(new T(std::forward<Args>(args)...));

```

```
4 }
5
6 // Utilisation :
7 auto ptr1 = make_unique<int>(42);
8 auto ptr2 = make_unique<std::string>("hello");
9 auto ptr3 = make_unique<Personne>("Alice", 25);
10 // Peu importe le nombre d'arguments !
```

8.4 Fold expressions (C++17)

En C++17, on peut simplifier avec les fold expressions :

```
1 template<typename... Args>
2 void print(Args... args) {
3     (std::cout << ... << args) << "\n";
4     // Fold expression : "..." signifie "pour chaque argument"
5 }
6
7 template<typename... Args>
8 auto sum(Args... args) {
9     return (... + args);
10    // Additionne tous les arguments
11 }
12
13 int main() {
14     print(42, " hello ", 3.14); // 42 hello 3.14
15     std::cout << sum(1, 2, 3, 4, 5) << "\n"; // 15
16 }
```

9 SFINAE et concepts (aperçu)

9.1 Le probleme

Parfois, on veut qu'un template ne marche QUE pour certains types.

```
1 template<typename T>
2 void afficherDouble(T value) {
3     std::cout << value * 2 << "\n";
4 }
5
6 afficherDouble(42);           // OK : 42 * 2 = 84
7 afficherDouble("abc");       // ERREUR ! On ne peut pas faire "abc" * 2
```

9.2 enable_if (C++11)

```
1 #include <type_traits>
2
3 // Seulement pour les types numeriques
4 template<typename T>
5 typename std::enable_if<std::is_arithmetic<T>::value>::type
6 afficherDouble(T value) {
7     std::cout << value * 2 << "\n";
8 }
9
10 afficherDouble(42);           // OK
11 afficherDouble(3.14);        // OK
12 // afficherDouble("abc");    // Erreur de compilation
```

9.3 Concepts (C++20)

Les concepts rendent ca beaucoup plus lisible :

```
1 // Definir un concept
2 template<typename T>
3 concept Numeric = std::is_arithmetic_v<T>;
4
5 // Utiliser le concept
6 template<Numeric T>
7 void afficherDouble(T value) {
8     std::cout << value * 2 << "\n";
9 }
10
11 // Ou encore plus simple :
12 void afficherDouble(Numeric auto value) {
13     std::cout << value * 2 << "\n";
14 }
```

10 Pieges et bonnes pratiques

10.1 Piege 1 : Templates dans les headers

IMPORTANT

Les templates doivent etre ENTIEREMENT dans les headers (.h ou .hpp).

Pourquoi ? Le compilateur a besoin de voir le code complet pour generer les instances.

MAUVAIS :

```

1 // fichier.h
2 template<typename T>
3 class MyClass {
4     void method();
5 };
6
7 // fichier.cpp
8 template<typename T>
9 void MyClass<T>::method() { /* ... */ }
10 // Ne marchera PAS !

```

BON :

```

1 // fichier.h
2 template<typename T>
3 class MyClass {
4     void method() {
5         // Implementation ici !
6     }
7 };

```

10.2 Piege 2 : Erreurs de compilation incomprehensibles

Les erreurs de template peuvent etre TRES longues et difficiles a lire.

Conseil : Regardez la PREMIERE erreur. Ignorez les suivantes qui sont souvent des consequences.

10.3 Piege 3 : Bloat de code

Chaque instantiation genere du code. Trop de templates peut augmenter la taille du binaire.

```

1 Stack<int> s1;           // Genere Stack<int>
2 Stack<double> s2;       // Genere Stack<double>
3 Stack<string> s3;       // Genere Stack<string>
4 // 3 classes differentes dans le binaire !

```

Solution : Utiliser des templates seulement quand necessaire.

10.4 Bonne pratique 1 : Utiliser auto

```

1 // Ancien style
2 std::vector<int>::iterator it = vec.begin();
3
4 // Nouveau style
5 auto it = vec.begin(); // Le compilateur deduit le type

```

10.5 Bonne pratique 2 : Template alias

```
1 // Au lieu de r p t e r :  
2 std::map<std::string, std::vector<int>> data1;  
3 std::map<std::string, std::vector<int>> data2;  
4  
5 // Definir un alias :  
6 template<typename T>  
7 using StringMap = std::map<std::string, T>;  
8  
9 StringMap<std::vector<int>> data1;  
10 StringMap<std::vector<int>> data2;
```

11 Quiz de validation

11.1 Questions

1. Qu'est-ce qu'un template ?
2. Quelle est la difference entre `typename` et `class` dans un template ?
3. Les templates sont-ils evalues a la compilation ou a l'execution ?
4. Peut-on avoir plusieurs parametres de type dans un template ?
5. Qu'est-ce que la specialisation de template ?
6. Pourquoi les templates doivent-ils etre dans les headers ?
7. Qu'est-ce qu'un template variadique ?
8. Quelle est la difference entre `max<int>()` et `max()` ?
9. Peut-on avoir des templates de valeurs (non-type templates) ?
10. Comment fonctionne `make_unique` en interne ?

11.2 Reponses

1. Un template est une recette pour generer du code qui marche avec n'importe quel type. C'est du code generique.
2. Aucune! `typename T` et `class T` sont exactement identiques dans un template.
3. A la COMPILATION. Le compilateur genere du code specifique pour chaque type utilise.
4. Oui! `template<typename T, typename U, typename V>`
5. La specialisation permet de creer une version speciale du template pour un type precis.
6. Car le compilateur a besoin de voir le code complet pour generer les instances.
7. Un template qui prend un nombre variable de parametres : `template<typename... Args>`
8. La premiere specifie explicitement le type (`int`). La seconde laisse le compilateur le deduire.
9. Oui! Exemple : `template<typename T, size_t N>` ou `N` est une valeur.
10. C'est un template variadique qui forward tous les arguments au constructeur avec `new`.

Fin de la Seance 5

Prochaine seance : Exceptions et Gestion d'Erreurs